

Garage Lions

Code Audit & Improvement Plan · 11 August 2026 · branch customlux-cabinets-board

Scope and method

Full read of the repository at `/var/www/garagelions` on branch `customlux-cabinets-board`, 11 August 2026. Static analysis of all Python and template sources, plus inspection of the live production error log, crontab, installed package set, and database migration state.

Shape of the codebase: roughly 15,800 lines of Python and 20,900 lines of templates across five Django apps. `panel/views.py` alone is 4,089 lines holding 151 view functions. There are **zero tests** — all four `tests.py` files are stubs, and the repository contains no `def test_` anywhere.

The code is genuinely well-written in places. CustomLux's access-control layer, the Plaid webhook JWT signature verification, the bank-account scoping helpers, and the `# -- Section --` organisation of the large view module are all solid work. The problems are concentrated in three areas: **authorization gaps in the panel app, the absence of any test or CI safety net on a checkout that is itself production, and operational hygiene.**

P0 — Security

Broken object-level authorization in `/panel/`

This is the most serious finding in the audit.

Panel views are protected by `@login_required` **only** — there is no per-object ownership check in the decorator layer. Read views compensate by scoping their queriesets through the `_filter_by_sp()` helper. But **24 endpoints look up objects by raw primary key with no scoping at all:**

L750	<code>ajax_estimate_update_header</code>	<code>Estimate</code>	<code>POST</code>
L821	<code>ajax_estimate_add_item</code>	<code>Estimate</code>	<code>POST</code>
L1014	<code>ajax_estimate_update_component</code>	<code>EstimateComponent</code>	<code>POST</code>
L1025	<code>ajax_estimate_delete_component</code>	<code>EstimateComponent</code>	<code>POST</code>
L1073	<code>ajax_estimate_template_save</code>	<code>Estimate</code>	<code>POST</code>
L1106	<code>ajax_estimate_template_apply</code>	<code>Estimate</code>	<code>POST</code>
L1142	<code>ajax_estimate_package_apply</code>	<code>Estimate</code>	<code>POST</code>
L1187	<code>ajax_estimate_template_delete</code>	<code>EstimateTemplate</code>	<code>POST</code>
L1197	<code>ajax_estimate_update_item</code>	<code>EstimateItem</code>	<code>POST</code>
L1252	<code>ajax_estimate_delete_item</code>	<code>EstimateItem</code>	<code>POST</code>
L1291	<code>ajax_estimate_move_items</code>	<code>Estimate</code>	<code>POST</code>
L1453	<code>transaction_edit</code>	<code>Transaction</code>	
L1464	<code>transaction_delete</code>	<code>Transaction</code>	

L2473	part_edit	Part	
L2597	ajax_template_delete	EstimateTemplate	POST
L2613	template_edit	EstimateTemplate	
L2635	ajax_template_add_item	EstimateTemplate	POST
L2695	ajax_template_update_item	EstimateTemplate	POST
L2723	ajax_template_delete_item	EstimateTemplate	POST
L2823	ajax_package_update	EstimatePackage	POST
L2840	ajax_package_delete	EstimatePackage	POST
L3038	task_create	TaskList	
L3052	task_toggle	Task	
L597	estimate_component_edit	EstimateComponent	

Stated concretely: `estimate_edit` correctly returns 404 when a salesperson opens another location's estimate. But a POST to `/panel/estimates/<that same pk>/ajax/delete-item/<n>/` succeeds. **The read wall is enforced; the write wall is not.** A salesperson assigned to sales point 3 can silently rewrite prices on, or delete line items from, any estimate in the company.

Transaction has no scoping field at all

`Transaction` (`panel/models.py:734`) carries no `sales_point` foreign key. Consequently `transaction_list` (L1358) applies no visibility filter whatsoever — every logged-in user sees the entire company ledger — and `transaction_delete` allows any of them to delete any row.

This one cannot be fixed in the view layer alone. It requires an additive schema change plus a backfill.

Secrets stored in plaintext

`PlaidItem.access_token` is a plain `CharField` (`panel/models.py:598`), and `GoogleCalendarCredential.access_token` is a plain `TextField`. Combined with the 20 MB of `pg_dump` output in `backups/`, a leaked database dump grants live bank-feed and calendar access. The existing "never commit backups/" rule addresses half of this exposure; encryption at rest addresses the other half.

Django is end-of-life and unpatched

The installed version is **Django 4.2.16**. Django 4.2 LTS reached end of extended support in April 2026, and 4.2.16 is roughly ten security releases behind even within the 4.2.x line. `pip list --outdated` reports 6.1 as current.

P1 — Correctness

Three divergent implementations of the same permission rule

`CLAUDE.md` explicitly warns that these must be kept in sync. They have already drifted, and there are three copies, not the two the guide describes:

- `_lead_queryset()` (`panel/views.py:71`) branches on **hardcoded role codes**: `pm.role == ProjectManager.TERRITORY_MANAGER`.
- `_visible_sp_ids()` delegating to `get_visible_sales_points()` (`account/models.py:468`) branches on **Role table flags**: `sees_all_locations` and `allows_multiple_locations`.
- `_visible_leads_qs()` (`panel/context_processors.py:5`) is a **third, independent copy** of the role-code version, duplicated so the sidebar badge can count new leads. It carries its own comment claiming it "matches `panel.views._lead_queryset`" — which is exactly the kind of promise that rots silently.

For the same user, leads, the sidebar badge, and customers/estimates/invoices can therefore resolve to different visibility sets. Create a new Role in the admin with `sees_all_locations=True` and any code other than `territory_manager`, and that person will see every customer but only their own leads — while the badge counts a third population. This is a live latent bug, not a theoretical one.

Notifications run synchronously on the public lead form

`home/views.py:343-347` fires four notification functions inline: multiple SMTP sends through SendGrid, a Twilio SMS, and a web push — all before the customer receives a response. There is no queue anywhere in the project; no Celery, no django-q, not even a background thread.

Two consequences. A slow SendGrid response hangs the highest-value form on the site. And because every failure is caught and swallowed internally, a lead can be created with nobody notified and no visible signal that anything went wrong.

Two management commands are never scheduled

`panel/management/commands/sync_bank_accounts.py` exists but the crontab contains exactly one entry — `send_followup_reminders`. Plaid balances therefore refresh only on webhook delivery or manual sync. `home/management/commands/send_stale_lead_reminders.py` is likewise unscheduled.

Uncommitted work mixes Plaid and CustomLux

The working tree carries 19 modified files spanning both features, against the standing rule never to mix them in one commit. `panel/views.py` (+880 lines) and `garagelions/settings/base.py` both contain hunks from each feature and need hunk-level staging.

P2 — Performance

The panel dashboard fires roughly 55 queries per load

In `panel/views.py:139`, the twelve-month loop issues two aggregates per month (24 queries), then a separate lead count per month (12 more), followed by about 15 further `.count()` and `.aggregate()` calls. On top of that, `BankAccount.pending_count` is a Python property that queries once per account.

The whole block collapses to about six queries using `TruncMonth` with `values().annotate()`.

The CustomLux board walks its projects three times

`_board_summary` is correctly prefetched at `customlux/views.py:293`. But the board-column loop immediately above it (L270-284) iterates `projects.order_by(...)` as a **separate, unprefetched queryset evaluation**, then reads `p.commission_amount` on every card — and each of those reads walks `change_orders`. The same queryset is evaluated a third time to produce `total_open`.

This is precisely the N+1 the function's docstring was written to avoid; it has simply moved one block upward.

Ten list views have no pagination

Including `transaction_list`, `part_list`, `customer_list`, `estimate_list`, `invoice_list`, and `project_list`. Each renders its full table today and gets monotonically slower forever.

A 431 MB uncompressed video in static/

`static/video/garagelions.mp4` is 431 MB. The `preload="none"` attribute spares the initial page load, but any visitor who presses play pulls the full 431 MB from the origin server. It is also why `staticfiles/` has grown to 877 MB.

P3 — Maintainability

`panel/views.py` at 4,089 lines is well past the point where its section comments still help navigation. It is approximately seven distinct modules in one file.

Templates hold **363 KB of inline JavaScript and CSS** with no build step: `estimates/edit.html` carries 52 KB inline, `parts/list.html` 40 KB, `cabinet_designer.html` 49 KB. None of it is cacheable, minifiable, or lintable.

`requirements.txt` pins 25 direct dependencies while the venv holds 62 packages, and **gunicorn is not listed at all**. Rebuilding an environment from `requirements.txt` produces something that cannot serve the site.

P4 — Operations

`django_errors.log` is **30 MB, unrotated, and carries no timestamps** — the `FileHandler` configured at `production.py:110` has no formatter attached. It is impossible to determine when any recorded error occurred, which is the main reason the log is not useful for debugging.

99.7% of that file is noise. Of 30,133 entries, 30,058 are `DisallowedHost` exceptions from scanners hitting the bare IP address, plus traffic for `mail.garagelions.com` and a stale `spao.me` vhost. Only **75 genuine HTTP 500 responses** are buried in there.

`logs/followups.log` stands at 8.4 MB, appended every single minute by cron, also unrotated.

The session table is never swept

`django_session` holds **10,816 live rows** against six user accounts. Django's `clearsessions` command is not in the crontab, so expired rows are never removed and the table only grows.

Some of those rows reference user IDs 10, 11 and 12 — accounts that no longer exist. Session rows are not foreign-keyed to the user table, so deleting a user leaves its sessions behind as undeletable orphans that every session lookup still has to page past.

The plan

Phase 0 — Untangle the working tree

Nothing else is safe to touch while 880 lines of uncommitted view code sit on disk in production. The tree now holds **four** unrelated concerns, so it needs four commits rather than two.

1. Run `git add -p panel/views.py garagelions/settings/base.py` and split the Plaid hunks from the CustomLux hunks.
2. Commit Plaid on its own: `panel/plaid_client.py`, `panel/models.py`, migrations 0019 through 0022, `templates/panel/bank/`, `panel/management/`, `requirements.txt`.
3. Commit CustomLux separately: `customlux/*` (excluding the `_username_for` change), `templates/customlux/*`, migration 0016.
4. Commit the audit report page: `panel/audit.py`, `templates/report.html`, `docs/AUDIT.md`, and the two-route hunk in `garagelions/urls.py`.
5. Commit the username rule: `account/models.py`, `account/admin.py`, `account/migrations/0014`, and the `_username_for` delegation in `customlux/views.py`.

6. Run `venv/bin/python manage.py check`, then restart gunicorn.

Estimated effort: 45 minutes.

Phase 1 — Close the authorization holes

Highest value work in this document.

7. Add `_owned_estimate(user, pk)`, `_owned_template(...)` and `_owned_package(...)` helpers alongside `_filter_by_sp`, then route all 22 estimate, template, package and part endpoints through them. Mechanical, roughly one hour, no behavioural change for legitimate users.
8. Add a nullable `Transaction.sales_point` column (additive and safe), backfill it from `project.sales_point` and `invoice.sales_point`, then wrap `transaction_list`, `transaction_edit` and `transaction_delete` in `_filter_by_sp`. **This step requires explicit sign-off** — the backfill is a live-database UPDATE.
9. Scope `task_create` and `task_toggle` to task lists the requesting user can actually see.

Phase 2 — Eliminate the permission-rule drift

10. Delete the role-code branches inside `_lead_queryset` and have it call `pm.get_visible_sales_points()` instead. One rule, one place. Small diff, large blast radius — schedule it immediately after Phase 3 provides test coverage.

Phase 3 — A minimal safety net

11. Write roughly 20 tests covering only what Phases 1 and 2 touch: "a salesperson at sales point 3 receives 404 for sales point 10's estimate, transaction and template," repeated per endpoint. Locking the fix in is the entire purpose of this suite.
12. Add a `Makefile` target or shell script chaining `check`, `migrate --check` and `test`, to be run before every gunicorn restart.

Phase 4 — Operational cleanup

Fast to implement, immediate relief.

13. Attach a formatter including `%(asctime)s` and switch to `RotatingFileHandler` in the `production.py` `LOGGING` dict.
14. Add an `nginx default_server` block returning 444 for unmatched Host headers, killing 30,000 log lines at the source. Additionally set the `django.security.DisallowedHost` logger to `CRITICAL`.

15. Truncate both existing logs and install `/etc/logrotate.d/garagelions`. Requires interactive `sudo`.
16. Add `sync_bank_accounts` hourly, `send_stale_lead_reminders` daily, and `clearsessions` daily to the crontab. The last one is what stops `django_session` growing without bound.
17. Emit `venv/bin/pip freeze > requirements.lock.txt`, keep `requirements.txt` as the curated human list, and add `unicorn` to it.

Phase 5 — Django upgrade

18. Upgrade 4.2.16 to the latest 4.2.x first. Patch-only, near-zero risk. Restart and verify.
19. Then plan 4.2 to 5.2 LTS as a separately tracked piece of work. With Phase 3's tests in place it stops being a leap of faith.

Phase 6 — Performance

20. Rewrite the dashboard's twelve-month loops using `TruncMonth` aggregation, taking roughly 55 queries down to about 6.
21. Fix the CustomLux board's unprefetched column loop; evaluate the `queryset` once into a list and reuse it.
22. Paginate the six unbounded list views.
23. Transcode `garagelions.mp4` to H.264 at 720p or 1080p, which lands around 15 to 30 MB, or move it to a CDN.

Phase 7 — Structural

Only once everything above is done.

24. Split `panel/views.py` into `panel/views/{leads,estimates,parts,bank,transactions,mobile,calendar}.py`. Pure moves, with imports re-exported from `__init__.py` so no URL configuration changes.
25. Encrypt `PlaidItem.access_token` and `GoogleCalendarCredential.access_token` at rest.
26. Extract the five largest inline `<script>` blocks into `static/js/`.

Recommended starting point

Phase 0 today, because the working tree is dirty in production. Then Phase 1 steps 7 and 9 — approximately one hour of mechanical, zero-risk edits that close 22 of the 24 authorization holes.

Step 8, the Transaction backfill, should be held until the live-database `UPDATE` has been reviewed and approved.